# Supersede: When Retrieval-Augmented Generation Cites the Obsolete Version

Measuring staleness, and a librarian agent that recovers document lineage

Carl Hatoum

Living draft, built from `results/` at revision `18a31e5+dirty` on 2026-11-05

**Abstract**

Knowledge bases accumulate documents that replace older ones, yet retrieval treats every version as equally valid. We build a benchmark from the IETF RFC index, whose "obsoletes" metadata gives a free, exact supersession graph over 9,852 documents and 1,543 edges. We show that lexical, dense and hybrid retrievers all rank the obsolete version first for a large share of queries, on title queries and on LLM-generated questions whose answer changed between versions, and that this staleness reaches generated answers. With lineage known, a simple promotion policy removes stale top results; without it, we must infer lineage. We describe a cascade: a cheap feature-based scorer for clear cases and a tool-using *librarian agent* (LangGraph, tools served over the Model Context Protocol, local open-weight models) for the uncertain ones. **This is a living draft**: each section states whether its numbers are final or preliminary.

## 1 Introduction

Enterprise and technical document collections are not static. Standards are revised, procedures are reissued, and policies are cancelled and replaced. The old version usually stays in the collection, and nothing in a retrieval-augmented generation (RAG) pipeline [1] tells the retriever or the generator which version is in force. The older document is often the better lexical and semantic match: it has the canonical title, it was cited more, and the newer one may only describe its changes.

We ask three questions.

1. **How often does retrieval surface the superseded version first?** (Section 4).
2. **Does this reach the generated answer, and can answer policies contain it?** (Section 5).
3. **When lineage is not recorded, can we recover it?** With a feature-based scorer, and with a tool-using agent on the cases the scorer is unsure about (Sections 6 and 7).

**Contributions.** (i) A reproducible staleness benchmark built from public metadata, with leakage-controlled settings for lineage inference. (ii) Retrieval- and answer-level staleness measurements across three retriever families and three answer policies. (iii) A cascade for lineage inference whose second stage is a librarian agent with explicit tools, evaluated where the first stage is uncertain. (iv) The tools are served over MCP, the same interface used in deployment.

## 2 Related work

[TODO: expand; verify every reference before submission.] Time-sensitive question answering studies questions whose answer changes over time [2, 3, 4]; we study the document side: which

| outcome | pairs |
|---|---|
| skipped_by_model | 224 |
| accepted | 69 |
| evidence_current_not_verbatim | 2 |
| question_leaks_reference | 2 |
| answer_from_silence | 1 |
| evidence_stale_not_verbatim | 1 |
| question_reveals_answer | 1 |

Table 1: Outcome of change-aware question generation per attempted edge.

version of a document is in force. Knowledge-conflict work examines how models resolve contradictions between parametric memory and context, or between passages [5, 6]; our conflicts have a ground-truth resolution given by the supersession graph. Our retrievers are BM25 [7], BGE dense embeddings [8] and reciprocal rank fusion [9]. We use an LLM judge [10] validated by agreement statistics [11], and a ReAct-style tool loop [12] for the agent.

## 3 Benchmark

[final for the RFC snapshot e1669ddd5a87]

**Corpus and lineage.** We parse the RFC index (`rfc-index.xml`, snapshot `e1669ddd5a87`, built 2026-10-30). Each document has a title, an abstract (for 9,145 of 9,852), keywords and a year. The index records *obsoletes* and *obsoleted-by* on both sides; we take an edge from either side, giving 1,543 supersession edges over 1,384 superseded documents, in 800 connected lineages (largest: 35 documents). *Updates* relations (2,162) are not supersession and are kept apart. The *current versions* of a document are the heads reached by following successors; a split can yield several.

**Splits.** Every query or pair belonging to one lineage component falls in the same split (dev or test, by a hash of the component, 30/70), so no lineage is seen on both sides.

**Title queries.** For each edge we form two queries, the old title and the new title, deduplicated per component: 2,246 queries (735 dev, 1,511 test). The correct result is any current version; any superseded member of the lineage is stale.

**Change-aware questions.** Title queries are artificial. For a seeded sample of 300 of 1,352 eligible edges, an LLM (prompt `q-v3`) writes a natural question whose correct answer *differs* between the old and new abstract, with both answers and verbatim evidence from each. Deterministic checks reject non-verbatim evidence, questions that name an RFC or reveal the answer, and answers argued from silence. 69 questions were accepted (23.0%); the model declined 224 pairs where it found no substantive change (Table 1).

## 4 Experiment 1: retrieval staleness

[final]

**Setup.** Retrievers: BM25 ($k_1$=1.2, $b$=0.75), dense `bge-small-en-v1.5` with exact cosine search, and their reciprocal rank fusion ($k$=60). Top-10. Metrics: *stale@1* (the first result is a superseded member of the query's lineage), *current@1*, *current@10* and MRR of the first current version, with 95% bootstrap intervals (1,000 resamples). Three post-retrieval policies use the *oracle* graph:

| retriever | query | policy | stale@1↓ | current@1↑ | current@10↑ | MRR↑ |
|---|---|---|---|---|---|---|
| bm25 | new title | `none` | 0.417 | 0.440 | 0.931 | 0.628 |
| | | `filter` | 0.000 | 0.750 | 0.931 | 0.821 |
| | | `promote` | 0.000 | 0.857 | 0.983 | 0.905 |
| | old title | `none` | 0.722 | 0.152 | 0.790 | 0.394 |
| | | `filter` | 0.000 | 0.568 | 0.790 | 0.651 |
| | | `promote` | 0.000 | 0.874 | 0.986 | 0.916 |
| dense | new title | `none` | 0.459 | 0.446 | 0.928 | 0.643 |
| | | `filter` | 0.000 | 0.803 | 0.928 | 0.855 |
| | | `promote` | 0.000 | 0.904 | 0.992 | 0.938 |
| | old title | `none` | 0.749 | 0.170 | 0.793 | 0.415 |
| | | `filter` | 0.000 | 0.620 | 0.793 | 0.686 |
| | | `promote` | 0.000 | 0.916 | 0.996 | 0.947 |
| hybrid | new title | `none` | 0.528 | 0.395 | 0.937 | 0.621 |
| | | `filter` | 0.000 | 0.809 | 0.937 | 0.861 |
| | | `promote` | 0.000 | 0.923 | 0.994 | 0.950 |
| | old title | `none` | 0.816 | 0.111 | 0.810 | 0.389 |
| | | `filter` | 0.000 | 0.615 | 0.810 | 0.689 |
| | | `promote` | 0.000 | 0.927 | 0.996 | 0.953 |

Table 2: Retrieval staleness on test-split title queries ($n$=727 new-title, 784 old-title). `filter` and `promote` use the oracle lineage graph.

| retriever | policy | stale@1↓ [95% CI] | current@1↑ |
|---|---|---|---|
| bm25 | `none` | 0.565 [0.449, 0.681] | 0.362 [0.261, 0.478] |
| | `filter` | 0.000 [0.000, 0.000] | 0.768 [0.667, 0.870] |
| | `promote` | 0.000 [0.000, 0.000] | 0.928 [0.870, 0.986] |
| dense | `none` | 0.551 [0.435, 0.667] | 0.304 [0.203, 0.420] |
| | `filter` | 0.000 [0.000, 0.000] | 0.681 [0.565, 0.783] |
| | `promote` | 0.000 [0.000, 0.000] | 0.841 [0.754, 0.913] |
| hybrid | `none` | 0.652 [0.536, 0.768] | 0.261 [0.159, 0.362] |
| | `filter` | 0.000 [0.000, 0.000] | 0.783 [0.681, 0.870] |
| | `promote` | 0.000 [0.000, 0.000] | 0.913 [0.841, 0.971] |

Table 3: Retrieval staleness on change-aware questions (all splits, $n$=69).

`none`; `filter`, which drops superseded documents; and `promote`, which replaces each superseded hit by its current version(s) at the same rank. The policies bound what perfect lineage can achieve.

**Results.** Table 2. On new-title queries, the superseded version is ranked first 0.417 of the time with BM25, 0.459 with dense retrieval and 0.528 with the hybrid: the better retriever is *more* stale, because it matches the topic better and the old document is the canonical match. On old-title queries stale@1 reaches 0.816. On change-aware questions (Table 3, $n$=69), the hybrid is stale at rank 1 0.652 [0.536, 0.768] of the time. With oracle lineage, `promote` raises hybrid current@1 to 0.923 (new titles) and 0.913 (questions).

## 5 Experiment 2: answer staleness

[final for $n$=69; human judge check pending]

| context | policy | stale answer↓ [95% CI] | current↑ | conflict flagged | abstain |
|---|---|---|---|---|---|
| standard | `naive` | 8.7% [2.9%, 15.9%] | 84.1% | 63.8% | 5.8% |
| | `latest_only` | 1.4% [0.0%, 4.3%] | 88.4% | 4.3% | 7.2% |
| | `show_conflict` | 0.0% [0.0%, 0.0%] | 94.2% | 82.6% | 4.3% |
| blind | `naive` | 21.7% [13.0%, 31.9%] | 65.2% | 42.0% | 13.0% |
| | `latest_only` | 1.4% [0.0%, 4.3%] | 81.2% | 4.3% | 14.5% |
| | `show_conflict` | 0.0% [0.0%, 0.0%] | 88.4% | 79.7% | 10.1% |

Table 4: Answer staleness on change-aware questions ($n$=69), hybrid retrieval, top-5 context.

**Setup.** For each change-aware question, the hybrid retriever returns its top-10 and the answerer (`claude-sonnet-5-5`) sees the top-5 under three policies: `naive` (as retrieved), `latest_only` (`promote` with the oracle graph, then keep current versions), and `show_conflict` (the `promote` order, with superseded documents kept and marked as replaced). A reference-based judge (`claude-haiku-5-5`) compares each answer with the reference current and stale answers and labels it *current*, *stale*, *mixed-flagged* (both, with the old one marked as outdated), *mixed-unflagged*, *abstain* or *other*. A *stale outcome* is *stale* or *mixed-unflagged*.

In the standard context each passage shows its year and number, which lets a strong answerer resolve versions on its own. The *blind* ablation removes years, masks RFC numbers and labels passages D1..Dk, as in collections without reliable metadata.

**Judge validity.** A second model (Claude Opus) labelled 100 answers blind to the judge's labels: Cohen's $\kappa$=0.85 (91 of 100 identical). This is model–model agreement, not human validation; a human spot check is pending (human $\kappa$=??, $n$=??).

**Results.** Table 4. With metadata visible, the naive policy yields a stale outcome 8.7% of the time; blind, 21.7% [13.0%, 31.9%]. With oracle lineage, `show_conflict` gives 0.0% stale outcomes and the most current answers (88.4% blind). Lineage therefore matters most precisely where metadata is missing, which motivates inferring it.

## 6 Inferring lineage: the cheap scorer

[final (librarian v0)]

**Leakage control.** RFC abstracts often state the lineage ("This document obsoletes RFC 2581"), and the lineage fields themselves are hidden from every method. Three settings: `declared` keeps the text as is; `masked` replaces every RFC number by a neutral token; `strict` also deletes sentences that declare a supersession. `strict` is the setting of interest for collections without any declaration.

**Candidates and features.** Candidate pairs are each document's 10 nearest neighbours by abstract embedding and by title embedding, ordered by publication date (120,390 pairs in `strict`; they contain 0.893 of the true edges). Features: title Jaccard and cosine, abstract cosine, year gap, same year, version cues, keyword Jaccard, inverse neighbour rank, and in `declared` a supersession cue mentioning the older document. A heuristic and a logistic regression trained on dev pairs (threshold tuned on dev F1) score each pair; each old document keeps its best successor above threshold.

**Results.** Table 5. Edge F1 falls from 0.663 (declared) to 0.539 (strict). Downstream, the inferred graph with `promote` closes 51% of the current@1 gap between no lineage and oracle

| setting | method | edge P | edge R | edge F1 | title queries: current@1 (gap) | questions: current@1 (gap) |
|---|---|---|---|---|---|---|
| declared | no lineage | – | – | – | 0.248 (0%) | 0.204 (0%) |
| | heuristic | 0.325 | 0.541 | 0.406 | 0.612 (54%) | 0.694 (69%) |
| | logreg | 0.775 | 0.578 | 0.663 | 0.660 (61%) | 0.694 (69%) |
| | oracle | – | – | – | 0.925 (100%) | 0.918 (100%) |
| masked | no lineage | – | – | – | 0.248 (0%) | 0.204 (0%) |
| | heuristic | 0.288 | 0.449 | 0.351 | 0.543 (44%) | 0.592 (54%) |
| | logreg | 0.593 | 0.481 | 0.531 | 0.594 (51%) | 0.551 (49%) |
| | oracle | – | – | – | 0.925 (100%) | 0.918 (100%) |
| strict | no lineage | – | – | – | 0.248 (0%) | 0.204 (0%) |
| | heuristic | 0.286 | 0.444 | 0.348 | 0.539 (43%) | 0.571 (51%) |
| | logreg | 0.596 | 0.492 | 0.539 | 0.594 (51%) | 0.551 (49%) |
| | oracle | – | – | – | 0.925 (100%) | 0.918 (100%) |

Table 5: Lineage inference on test-split pairs, and downstream current@1 with `promote` over the inferred graph (gap closed between no lineage and oracle in brackets).

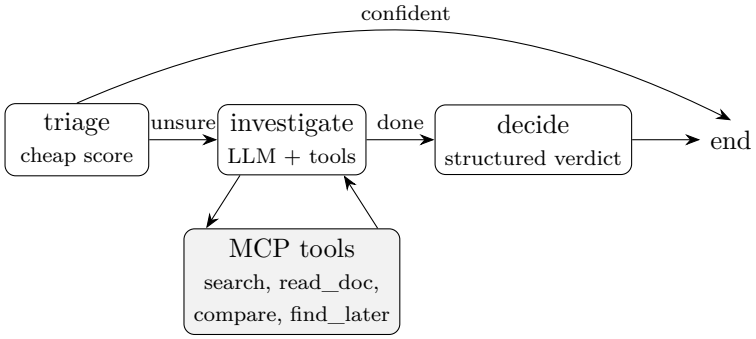



Figure 1: The librarian agent. Only pairs in the cheap scorer's uncertain band reach the LLM.

lineage on title queries in `strict` (61% in `declared`). Half the gap remains, and the scorer's errors concentrate in a band of high but uncertain scores: this is where the agent is used.

# 7 The librarian agent

[preliminary: $n$=20 pairs per run]

**Design.** The agent decides whether a newer document supersedes an older one. It is a LangGraph state machine (Figure 1): *triage* accepts or rejects pairs on which the cheap scorer is confident, *investigate* is a tool-calling loop (at most 8 steps), and *decide* returns a structured verdict (*supersedes*, *amends*, *conflicts*, *unrelated*) with a confidence and a rationale. The tools are served by an MCP server, the same one used in deployment, and never expose lineage fields: `search(query)`; `read_doc(id)` (title, year, keywords, abstract); `compare(a, b)` (both documents, word overlaps, which is older); and, from prompt v2, `find_later(id)`, the documents on the same topic published the same year or later, i.e. the candidate replacements. Models run locally through Ollama: Qwen 3.5 (9.7B) and Ministral 3 (8B), at temperature 0. Prompt v2 also asks the agent to compare first, then look for later versions, to distinguish a successor from a companion document, and gives a confidence rubric.

**Evaluation protocol.** The agent only sees pairs whose cheap-scorer probability falls in the band [0.8, 0.98) (`strict` setting, test split). From that band we draw a seeded sample with equal numbers of true edges and non-edges, and compare the agent with the cheap scorer's own decision (probability $\geq$ its dev threshold 0.94) *on the same pairs*. We report true-positive and

| truth | method | prompt | TPR | TNR | bal. acc. [95% CI] | est. prec. |
|---|---|---|---|---|---|---|
| strict | cheap scorer (v0) | – | 0.40 | 0.90 | 0.65 [0.50, 0.85] | 0.40 |
| | agent: `ministral-3` | v2 | 0.90 | 0.60 | 0.75 [0.55, 0.90] | 0.27 |
| | agent: `qwen3.5-hermes` | v1 | 1.00 | 0.70 | 0.85 [0.70, 0.95] | 0.35 |
| | agent: `qwen3.5-hermes` | v2 | 0.90 | 0.90 | 0.90 [0.75, 1.00] | 0.60 |
| lineage | cheap scorer (v0) | – | 0.31 | 0.86 | 0.58 [0.40, 0.74] | 0.41 |
| | agent: `ministral-3` | v2 | 0.85 | 0.71 | 0.78 [0.56, 0.96] | 0.49 |
| | agent: `qwen3.5-hermes` | v1 | 0.92 | 0.86 | 0.89 [0.71, 1.00] | 0.68 |
| | agent: `qwen3.5-hermes` | v2 | 0.77 | 1.00 | 0.88 [0.73, 1.00] | 1.00 |

Table 6: Agent vs. cheap scorer on balanced samples from the uncertain band, `strict` setting. Estimated precision uses the band's real positive rate.

true-negative rates, balanced accuracy with a stratified bootstrap 95% interval, and an estimated precision at the band's real positive rate (0.14 strict), since the sample is balanced and the band is not. Two ground truths: *strict* (the index records the edge directly) and *lineage*, which also accepts a later version through a chain (A→B→C makes "C supersedes A" correct for a reader; band positive rate 0.25).

**Results so far.** Table 6. On the same 20 pairs, the cheap scorer reaches balanced accuracy 0.65 [0.50, 0.85] (strict truth). The Qwen agent with prompt v1 reaches 0.85 [0.70, 0.95], but it never searched (every trace is read, read, compare) and accepted same-title pairs too readily (TNR 0.70). With prompt v2 and `find_later`, it looks for later versions on every pair and reaches 0.90 [0.75, 1.00] (TNR 0.90). Ministral 3 with the same prompt reaches 0.75 [0.55, 0.90] (TNR 0.60): it made a single `compare` call on every pair and never used `find_later`, so it decides from titles and dates, and most of its errors are same-title pairs accepted as replacements. It also returned confidence 0 with two verdicts, i.e. it does not use the confidence field as intended. Following the investigation procedure is therefore model-dependent, which matters for a cheap local deployment. The intervals overlap: these runs show the direction, not a finding. Runs on $n \geq 100$ pairs are planned.

## 8 Limitations and next steps

- One domain so far (RFCs). RFC titles and abstracts are short; full texts and other collections (e.g. aviation advisory circulars) are planned.
- Only 69 change-aware questions; some references point to a new version that is itself superseded (to be fixed by generating from lineage heads only).
- The answer judge is validated against a second model, not yet humans.
- Agent results are on 20 pairs per run; the uncertain band has a low positive rate, so precision at deployment is the quantity to watch.
- Downstream impact of the agent's edges (gap closed) is not yet measured.
- Deployment (MCP and A2A interfaces, container on Cloud Run) is in progress and will be described with latency and cost.

## Reproducibility

Code, data build and every result file are in the repository; this PDF is built by `make paper`, which regenerates all numbers from `results/`. LLM responses for question generation, answering

and judging are cached in SQLite, so `make reproduce` rebuilds them offline. Agent decisions are logged per pair, with tool traces. The benchmark (corpus, lineage, title queries, change-aware questions and lineage-inference candidate pairs, with lineage-level splits) is packaged as a Hugging Face dataset, `hatoum/supersede-rfc` (private until release). RFC abstracts are shipped verbatim, as the IETF Trust Legal Provisions allow unmodified, attributed excerpts but not modified text; the `masked` and `strict` views are derived locally by a script shipped with the data.

# A Provenance

| section | source (under `results/` unless stated) |
|---|---|
| Retrieval staleness (title queries) | `rfc_staleness_test.json` |
| Retrieval staleness (change-aware questions) | `rfc_questions_all.json` |
| Question generation | `data/rfc/questions_manifest.json` |
| Answer staleness (standard) | `rfc_answer_staleness_hybrid.json` |
| Answer staleness (blind) | `rfc_answer_staleness_hybrid_blind.json` |
| Judge agreement (model rater) | `rfc_answer_staleness_hybrid_labels_claude-opus-5-5.csv` |
| Lineage inference (librarian v0) | `rfc_lineage_inference.json` |
| Agent: qwen3.5-hermes, agent-v1 | `agent/rfc_agent_strict_ollama-edtorre-qwen3-5-hermes_agent-v1.json` |
| Agent: qwen3.5-hermes, agent-v2 | `agent/rfc_agent_strict_ollama-edtorre-qwen3-5-hermes_agent-v2.json` |
| Agent: ministral-3, agent-v2 | `agent/rfc_agent_strict_ollama-ministral-3-8b_agent-v2.json` |

# B Agent runs

| model | prompt | setting | n | failed | steps | s/pair | labels |
|---|---|---|---|---|---|---|---|
| `qwen3.5-hermes` | agent-v1 | strict | 20 | 0 | 3.0 | 122 | amends 2, supersedes 13, unrelated 5 |
| `qwen3.5-hermes` | agent-v2 | strict | 20 | 0 | 3.0 | 118 | amends 2, supersedes 10, unrelated 8 |
| `ministral-3` | agent-v2 | strict | 20 | 0 | 2.0 | 69 | amends 2, supersedes 13, unrelated 5 |

# C Example agent traces

### `qwen3.5-hermes`, agent-v1

- **correct, supersedes**: `RFC332 -> RFC342` (gold supersedes; verdict `supersedes`, confidence 0.95).
  → `read_doc({"doc_id": "RFC332"})`
  → `read_doc({"doc_id": "RFC342"})`
  → `compare({"doc_a": "RFC332", "doc_b": "RFC342"})`
  *Both documents share the exact same title ('Network Host Status') and year (1972). In technical standards collections, when two documents with identical titles exist but are distinguished as 'older' and 'newer' (even if the publication year is the same or very close), the newer identifier typically represents a revision or replacement of the older one. The comparison tool confirmed they cover the same topic. Since RFC342 is designated as the 'NEWER' document in the pair, it supersedes RFC332.*

- **correct, not**: `RFC4145 -> RFC4572` (gold not; verdict `unrelated`, confidence 0.95).
  → `read_doc({"doc_id": "RFC4145"})`
  → `read_doc({"doc_id": "RFC4572"})`
  → `compare({"doc_a": "RFC4145", "doc_b": "RFC4572"})`
  *RFC4145 (2005) defines media transport over plain TCP, while RFC4572 (2006) defines media transport over TLS. Although both use SDP and share similar attribute names ('setup', 'connection'), they address different underlying protocols (TCP vs. TLS). RFC4572 does not replace or supersede RFC4145; instead, they are complementary standards for different*

*security requirements. Therefore, they are unrelated in the sense that neither supersedes the other.*

- **error**: `RFC266 -> RFC370` (gold not; verdict `supersedes`, confidence 0.95).
  → `read_doc({"doc_id": "RFC266"})`
  → `read_doc({"doc_id": "RFC370"})`
  → `compare({"doc_a": "RFC266", "doc_b": "RFC370"})`
  *The documents share identical titles ('Network host status' vs 'Network Host Status') and cover the same topic. The newer document (RFC370, 1972) is a direct update to the older one (RFC266, 1971). In technical standards, when a new version with the same title replaces an older one, it supersedes the previous version.*

### `qwen3.5-hermes`, agent-v2

- **correct, supersedes**: `RFC3016 -> RFC6416` (gold supersedes; verdict `supersedes`, confidence 0.95).
  → `compare({"doc_a": "RFC3016", "doc_b": "RFC6416"})`
  → `find_later({"doc_id": "RFC3016", "k": 5})`
  *RFC6416 explicitly identifies itself as a 'necessary revision' of RFC3016 in its abstract. It states that it updates and completes the specification while discussing backward compatibility with the older document. The scope (RTP Payload Format for MPEG-4 Audio/Visual Streams) is identical, and the newer document corrects misalignments found in the older one, which is the definition of superseding in this context.*

- **correct, not**: `RFC4145 -> RFC4572` (gold not; verdict `unrelated`, confidence 0.95).
  → `compare({"doc_a": "RFC4145", "doc_b": "RFC4572"})`
  → `find_later({"doc_id": "RFC4145", "k": 5})`
  *RFC4145 defines media transport over plain TCP, while RFC4572 defines media transport over TLS (Transport Layer Security). Although both are SDP extensions for connection-oriented transport, they address different underlying protocols with different security properties. One does not replace the other; instead, they serve as companion specifications allowing implementers to choose between unencrypted and encrypted transport mechanisms based on their needs. The fact that RFC8122 (a later update to the TLS specification) exists further confirms that RFC4572 was an early version of the TLS SDP spec, not a successor to the plain TCP spec.*

- **error**: `RFC332 -> RFC342` (gold supersedes; verdict `unrelated`, confidence 0.95).
  → `compare({"doc_a": "RFC332", "doc_b": "RFC342"})`
  → `find_later({"doc_id": "RFC332", "k": 5})`
  *Both documents share the exact same title ('Network Host Status') and were published in the same year (1972). The comparison tool indicates no abstract overlap (both lack abstracts) but 100% title word overlap. The 'find_later' query for RFC332 returned other RFCs from 1972, none of which were RFC342, suggesting they are not in a direct versioning chain where one replaces the other. Documents with identical titles from the same era often represent companion documents (e.g., different profiles or specific aspects) rather than a successor relationship. Since neither clearly restates and revises the scope of the other to act as a replacement, and they appear to be parallel works on the same topic, they are classified as unrelated in terms of supersession.*

### `ministral-3`, agent-v2

- **correct, supersedes**: `RFC3016 -> RFC6416` (gold supersedes; verdict `supersedes`, confidence 0.95).
  → `compare({"doc_a": "RFC3016", "doc_b": "RFC6416"})`

*RFC6416 explicitly states it is a 'necessary revision' of RFC3016, summarizes changes, and addresses backward compatibility issues. The scope remains identical (RTP payload formats for MPEG-4 Audio/Visual streams), and the newer document restates/revises the older one's content while clarifying limitations and correcting misalignments. This is a direct replacement, not merely an extension or companion document.*

- **correct, not**: `RFC4145 -> RFC4572` (gold not; verdict `amends`, confidence 0.9).
  → `compare({"doc_a": "RFC4145", "doc_b": "RFC4572"})`
  *RFC4572 explicitly states it 'extends and updates' a prior reference (likely RFC4145) by adding TLS security to TCP-based media transport in SDP. The newer document introduces a new protocol identifier ('TCP/TLS') and a fingerprint attribute, but it does not restate or revise the entire scope of RFC4145. Instead, it adds a new mechanism (TLS) to the same transport framework. The older document remains valid for plain TCP transport, and the newer one complements it rather than replacing it entirely. This is a clear case of amendment, not supersession.*

- **error**: `RFC332 -> RFC342` (gold supersedes; verdict `unrelated`, confidence 0.0).
  → `compare({"doc_a": "RFC332", "doc_b": "RFC342"})`
  *RFC332 and RFC342 share the same title and year (1972), meaning neither is newer than the other. Since supersession requires a later document to replace an earlier one, they cannot be in a superseding relationship. The lack of a chronological gap or explicit revision history makes this a definitive conclusion.*

# References


[1] P. Lewis et al. Retrieval-augmented generation for knowledge-intensive NLP tasks. *NeurIPS*, 2020.

[2] W. Chen, X. Wang, W. Y. Wang. A dataset for answering time-sensitive questions. *NeurIPS Datasets and Benchmarks*, 2021.

[3] J. Kasai et al. RealTime QA: What's the answer right now? *NeurIPS Datasets and Benchmarks*, 2023.

[4] B. Dhingra et al. Time-aware language models as temporal knowledge bases. *TACL*, 2022.

[5] S. Longpre et al. Entity-based knowledge conflicts in question answering. *EMNLP*, 2021.

[6] J. Xie et al. Adaptive chameleon or stubborn sloth: Revealing the behavior of large language models in knowledge conflicts. *ICLR*, 2024.

[7] S. Robertson, H. Zaragoza. The probabilistic relevance framework: BM25 and beyond. *Foundations and Trends in IR*, 2009.

[8] S. Xiao et al. C-Pack: Packaged resources to advance general Chinese embedding. arXiv:2309.07597, 2023.

[9] G. V. Cormack, C. L. A. Clarke, S. Büttcher. Reciprocal rank fusion outperforms Condorcet and individual rank learning methods. *SIGIR*, 2009.

[10] L. Zheng et al. Judging LLM-as-a-judge with MT-Bench and Chatbot Arena. *NeurIPS Datasets and Benchmarks*, 2023.

[11] J. Cohen. A coefficient of agreement for nominal scales. *Educational and Psychological Measurement*, 1960.

[12] S. Yao et al. ReAct: Synergizing reasoning and acting in language models. *ICLR*, 2023.